

TEMATICA: AI & Software Development

El AI Orchestrator

Cómo el SDLC debe cambiar

El ciclo de desarrollo asume que los humanos escriben el código.
Eso ya dejó de ser cierto.

Carlos Gallardo
cgallardo@cleveritgroup.com

DEVOPSDAYS.PE

27 - 28 AGO 2026





El punto de partida

El SDLC que conocemos tiene un supuesto oculto

Plan → Code → Build → Test → Release → Operate

todas estas fases asumen que **un humano escribe cada línea**



La velocidad

Un agente genera en minutos lo que tomaba días. El cuello de botella ya no es escribir.



La cantidad

No es un agente: son equipos de agentes trabajando en paralelo sobre el mismo repo.



El control

Nadie diseñó el ciclo para revisar, versionar y gobernar lo que producen los agentes.



El cuello de botella se movió

ANTES

Escribir código era lo caro

- Generar código = horas/días
- Revisión humana proporcional
- El repo cambia despacio
- El contexto vive en la cabeza del dev



AHORA

Gobernar el contexto es lo caro

- Generar código = minutos
- Revisión humana no escala
- El repo cambia por muchos agentes
- El contexto debe ser un artefacto versionado



El cambio de rol

El ingeniero ya no escribe. Orquesta.

El **trabajo deja de ser producir líneas** y pasa a ser **dirigir un equipo de agentes**: darles contexto correcto, verificar lo que devuelven, y cerrar el ciclo con evidencia.

● De escribir código → **a definir el contexto que lo produce**

● De revisar PRs → **a orquestar especialistas por dominio**

● De confiar en el output → **a exigir evidencia de ejecución**



AI Orchestrator

PO

TL

BE

FE

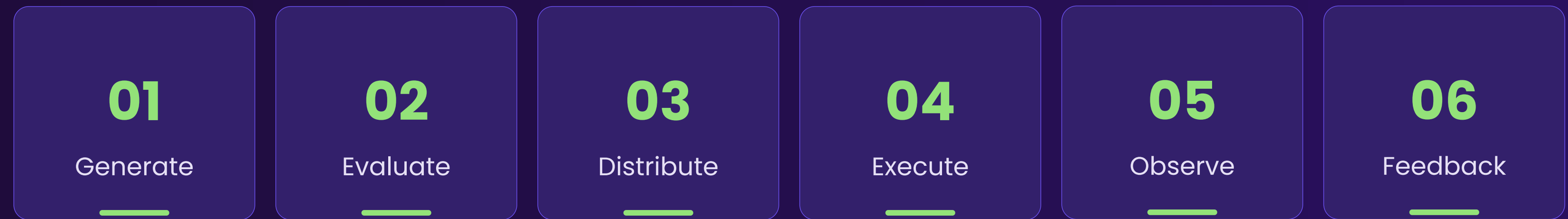
QA

un equipo de agentes especialistas



CDLC: Context Development Life Cycle

Si los agentes escriben el código, el artefacto que gobernamos ya no es el código: es el contexto. El CDLC es el ciclo de vida de ese contexto — seis fases para producirlo, verificarlo, distribuirlo y corregirlo.





El ciclo de vida del contexto

01 **Generate**

Los agentes producen contexto: paquetes, versiones, decisiones.

02 **Evaluate**

Se verifica el contexto antes de usarlo. (Hoy: falta)

03 **Distribute**

Port sirve contexto y config a cada workspace.

04 **Execute**

Los agentes ejecutan rutinas sobre el repo real.

05 **Observe**

Telemetría OTel: trazas, spans, evals por dominio.

06 **Feedback**

La evidencia vuelve al Curator para corregir. (Hoy: falta)



El corazón del problema

Dos brechas. La misma, vista dos veces.

● Evaluate – la brecha de entrada

Nada verifica el contexto antes de distribuirlo.

El Curator escribe a ciegas y publica sin red. Si el contexto está mal, todos los agentes heredan el error.

● Feedback – la brecha de salida

Nada devuelve evidencia de ejecución para corregirlo.

No hay loop de retorno hacia el Curator. Los errores de ejecución nunca regresan al contexto que los causó.



Ambas se cierran con el mismo insumo: telemetría de contexto.
Etiqueta cada traza OTel con el paquete y versión que la alimentó.



La arquitectura

Cómo se ve resuelto



LO QUE LA ARQUITECTURA YA RESUELVE



Context manager

Punto único de distribución de contexto y config.



Equipo agnóstico

PO · TL · BE · QA — sin casarse con un vendor.



OTel nativo

Telemetría de ejecución exportada de fábrica.



Demo en vivo

Covenant: El CDLC, construido y funcionando

● covenant.uno



Teams de agentes

Especialistas por rol (PO, TL, BE, FE, QA) que puedes activar y orquestar.



Workspaces por dominio

covenant · loyalty — cada uno con su contexto aislado.



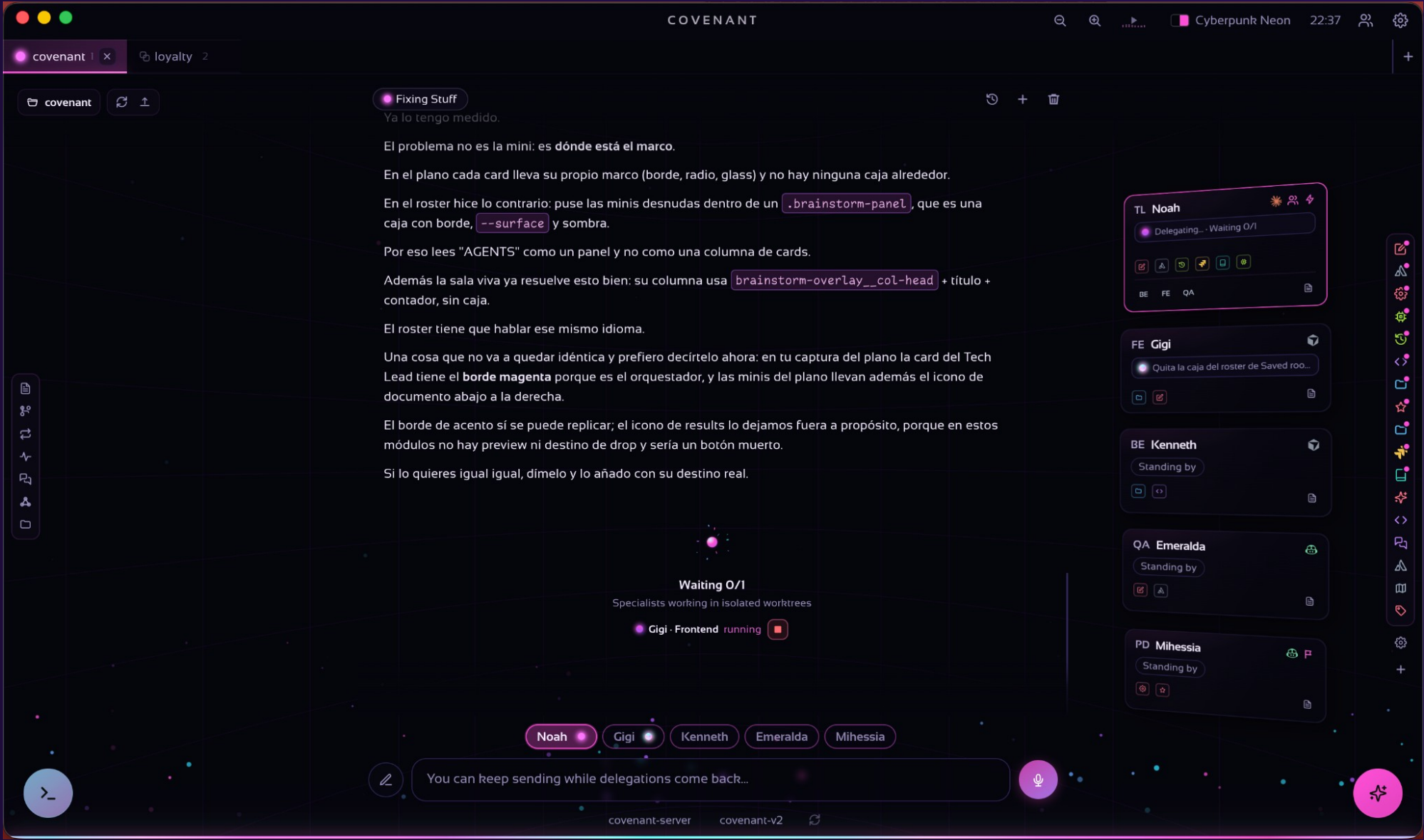
Self-update & Loop

Los agentes mantienen su propio contexto; el ciclo se retroalimenta.



DEMO: La aplicación

Un orquestador, no un chat



Workspaces

Covenant · loyalty como pestañas de dominio.

Equipo lateral

Cada agente (TL, FE, BE, QA, PO) con su propio contexto cargado.

Standing by

Los especialistas esperan ser orquestados, no responden en desorden.

Self-update

El agente mantiene y versiona su contexto de forma autónoma.



Qué significa para ti

Qué cambia en tu trabajo mañana



Trata el contexto como código

Versiónalo, revísalo y haz PRs de contexto — igual que del código.



Construye tu Eval Gate

No distribuyas contexto que no pasó una verificación. Cierra la brecha de entrada.



Cierra el loop con telemetría

Etiqueta cada traza con su paquete y versión. Devuelve evidencia al que lo escribió.



Vuélvete Orchestrator

Tu valor ya no es teclear rápido: es dirigir bien un equipo de agentes.



El SDLC no se muere **Se convierte en el CDLC.**

El código lo escriben los agentes. Nuestro trabajo es gobernar el contexto que los guía — y cerrar las dos brechas que hoy lo dejan a ciegas.



Gracias

Conectemos



Información de contacto

Email: carlos.gallardo@raven.inc / cgallardo@cleveritgroup.com

GitHub: [@karluiz](https://github.com/karluiz)
Linkedin: [/in/cpgallardo](https://www.linkedin.com/in/cpgallardo)